

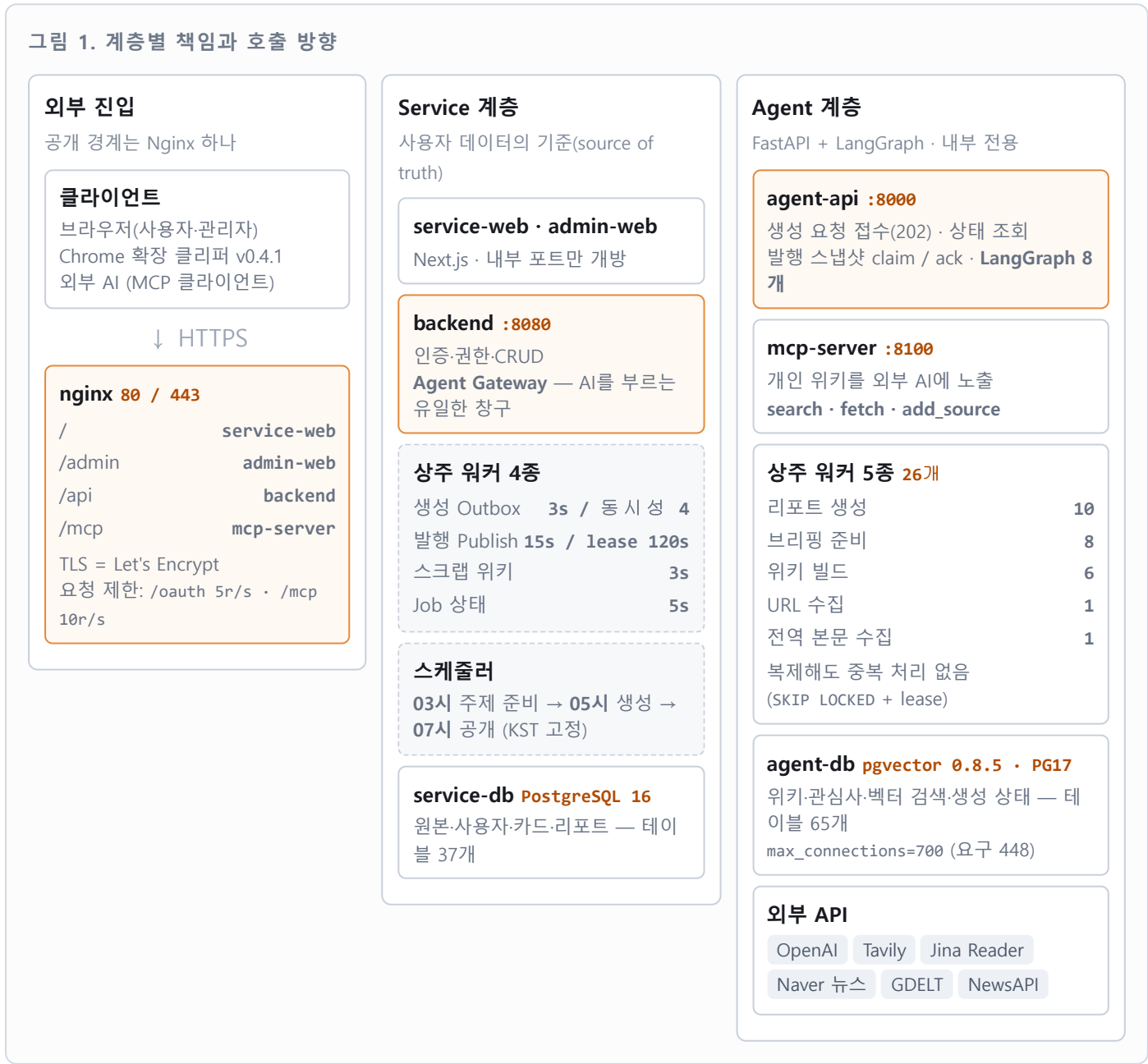
시스템 아키텍처 및 데이터 설계

통합 프로젝트 필수 산출물 03(시스템 아키텍처) · 04(데이터 설계도) | 기준일 2026-08-13 | 대상 브랜치 main

이 문서의 모든 수치는 코드에서 직접 확인한 값입니다. 근거는 bambi-build/docker-compose.yml · nginx/routes.conf · service-api/application.yml · Flyway 마이그레이션 · agent-api/database/migrations 이며, 항목마다 파일과 행 번호를 함께 적었습니다. 서버 IP·도메인·비밀값은 표기하지 않았습니다.

1. 전체 시스템 아키텍처

단일 VM 위에서 Docker Compose로 운영합니다. 외부에 열린 문은 **Nginx** 하나이며, 경로에 따라 네 개의 상위 서비스로 갈라집니다. AI 처리 계층(Agent)은 전부 내부 전용입니다.



구조를 지탱하는 규칙 3개

- ① 프론트엔드는 Agent를 직접 부르지 않는다 — 인증과 원본이 Service에 있기 때문입니다.
- ② Agent는 service-db를 건드리지 않는다 — 접속 정보 자체를 갖고 있지 않아 **위반이 물리적으로 불가능**합니다.
- ③ 완성물은 Service 워커가 당겨온다(Pull) — Agent에서 Service로 향하는 화살표가 없는 것이 설계입니다.

컨테이너 구성

Compose 서비스 정의는 **15종**이고, 이 중 `certbot · agent-db-init` 는 `profiles: ["ops"]` 라 평시에 뜨지 않습니다. 워커 replica를 펼치면 **상시 기동 컨테이너는 34개**입니다.

컨테이너	이미지	상시 수	외부 노출
nginx	nginx:1.27-alpine	1	80, 443 (유일한 공개 경계)
service-web	GHCR bambi-service-web	1	내부 3000
admin-web	GHCR bambi-admin-web	1	내부 3000 (basePath=/admin)
backend	GHCR bambi-service-api	1	내부 8080
postgres (service-db)	postgres:16	1	내부 5432
agent-db	pgvector:0.8.5-pg17	1	⚠ 호스트 포트 개방
agent-api	GHCR bambi-agent-api	1	⚠ 호스트 8000 개방
mcp-server	동일 이미지 · 다른 진입점	1	내부 8100
agent-worker-report	동일 이미지	10	없음
agent-worker-briefing	동일 이미지	8	없음
agent-worker-wiki	동일 이미지	6	없음
agent-worker-url	동일 이미지	1	없음
agent-worker-global-content	동일 이미지	1	없음
certbot · agent-db-init	—	0	profiles: ops — 수동 실행
합계		34	

한 이미지, 여섯 역할. `bambi-agent-api` 이미지 하나를 API-MCP 서버·워커 5종·DB 초기화까지 `command` 만 바꿔 재사용합니다. 빌드와 배포가 한 번으로 끝나고, 버전 불일치가 생길 여지가 없습니다.

2. 리포트 생성 데이터 흐름

메시지 브로커 없이 **DB만으로** at-least-once 전달과 멍등을 만듭니다. 사용자 요청과 아침 스케줄이 같은 경로를 씁니다.

그림 2. 요청 접수부터 카드 노출까지

사용자 → 웹 "지금 생성" 클릭 `POST /api/reports/generate`

backend Outbox 테이블에 INSERT하고 202만 응답

카드를 기다리지 않습니다. 같은 트랜잭션에 넣으므로 "저장은 됐는데 요청이 유실"되는 경우가 원천 차단됩니다.

Outbox 워커 3초 폴링 · 동시성 4 · lease 120초로 claim → `POST /internal/v1/users/{id}/generations`

실패하면 지수 백오프(5초~300초)로 최대 8회 재시도합니다.

agent-api Job 등록 후 202 + `job_id` 반환 → backend가 진행 상태를 갱신

리포트 워커 ×10 FOR UPDATE SKIP LOCKED로 Job 점유 → LangGraph 실행 → 결과를 `publish_snapshots`에 ready로 적재

워커를 몇 개로 늘려도 같은 Job을 두 번 잡지 않습니다.

Publish 워커 15초 폴링으로 완성물을 당겨온다(claim, lease 120초) → `reports + cards UPSERT` → ack

항목별 독립 트랜잭션이라 한 건이 실패해도 나머지가 함께 롤백되지 않습니다.

화면 `GET /api/reports/pending` 폴링 → "생성 중" 슬롯이 카드로 교체

시간으로 추측해 닫지 않고, 발행 데이터의 요청 식별자로 짝지어 닫습니다.

먹등이 3겹입니다. ① 요청 먹등기 — 생성 요청 id를 먹등기에서 결정적으로 파생시켜 버튼을 연타해도 1건입니다. ② Job 점유 — SKIP LOCKED + lease. ③ 발행 저장 — (`userId`, `external_content_id`, `external_version`) UPSERT, 유니크 충돌은 "이미 발행됨"으로 간주합니다. 그래서 워커 수를 1에서 10으로 늘리면서 애플리케이션 코드를 한 줄도 고치지 않았습니다.

아침 브리핑은 시각을 셋으로 나눴습니다

시각	하는 일	왜 이 시각인가
03:00	주제 선정과 근거 자료 준비	발화 시점에 실시간 수집을 하면 주제당 44~68초가 그대로 지연이 됩니다. 미리 끝내 둡니다.
05:00	리포트 생성 요청을 Outbox에 적재	생성에는 시간이 걸립니다. 07시에 시작하면 사용자가 빈 화면을 봅니다.
07:00	일괄 공개	그냥 앞당기면 먼저 만들어진 사람이 먼저 받습니다. 만드는 시각과 보이는 시각을 분리해 전원이 동시에 받습니다.

근거 — `docker-compose.yml:88~96`, `application.yml app.scheduler.generation.cron`은 `zone="Asia/Seoul"`로 고정해 컨테이너 TZ와 무관하게 항상 KST로 해석됩니다.

3. 데이터베이스 — 두 DB의 물리 분리

	service-db	agent-db
엔진	PostgreSQL 16	pgvector 0.8.5 / PostgreSQL 17
성격	원본·최종 사용자 노출 데이터	AI 파생물·임베딩·RAG·운영 로그
마이그레이션	Flyway 30개 (ddl-auto: validate)	SQL 32개 + 자체 러너
테이블	37개 (service 스키마)	65개 (관리 테이블 1개 포함 → 업무 64개)
접근 주체	Spring backend만	agent-api · 워커 · mcp-server만

왜 나뉘나 — ① 트랜잭션 처리와 벡터 검색은 워크로드가 다릅니다. ② LLM 팀이 스키마를 독립적으로 바꿀 수 있어야 서로를 막지 않습니다. ③ 팀 경계를 인프라 수준에서 강제합니다.

선언이 아니라 구조로 막았습니다. agent 컨테이너 환경변수에는 AGENT_DATABASE_URL 만 있고 service-db 접속 정보가 없습니다. 규칙을 어기려 해도 불을 주소를 모릅니다.

3.1 service-db ERD — 사용자에게 보이는 데이터

그림 3. 핵심 엔티티 (전체 37개 중 관계가 중요한 것)

users id PK public_id UK 대외 노출용 UUID email UK password_hash BCrypt username UK default_card_visibility 기본 PUBLIC (V31) report_ready_notification change_history_enabled deleted_at soft delete	cards id PK public_id UK user_id FK report_id FK title why_for_you visibility PRIVATE PUBLIC external_content_id external_version 둘이 발행 맥등키	reports id PK public_id UK user_id FK body 본문은 리포트에만 report_type cover_image_url change_history_enabled 본문 렌더링 형식 신호 external_content_id external_version	generation_pendings id PK 맥등키에서 파생 user_id FK idempotency_key UK report_type topic agent_job_id status card_public_id 완성 카드와 연결
bookmarks id PK user_id FK url · content summary Agent 요약물 저장 status PROCESSING DONE FAILED deleted_at	interests id PK user_id FK name source USER INFERRED (AI가 추론한 것 구분)	roles · user_roles 역할 기반 접근 제어 role_id FK user_id FK USER / ADMIN 분리. 관리자 화면은 ADMIN만.	ai_request_logs ai_response_logs endpoint request_body jsonb status_code response_body jsonb latency_ms 요청·응답을 1:N으로 짝지어 적재

관계 users 1:N bookmarks · interests · cards · reports · generation_pendings · follows · scraps · notifications · user_roles

cards 1:N card_sources(출처) · likes · comments · cards N:1 reports · reports 1:N report_citations(인용) · ai_request_logs 1:N ai_response_logs

설계 포인트 다섯

결정	내용과 이유
카드 ≠ 리포트	카드는 피드용 요약, 리포트는 본문입니다. 본문을 <code>reports</code> 에만 두어 피드를 조회할 때 본문을 읽지 않습니다 . 목록 응답이 가벼워집니다.
대외 ID 분리	밖으로는 <code>public_id</code> (UUID)만 내보내고 내부 순번 <code>id</code> 는 노출하지 않습니다. 순차 <code>id</code> 추측으로 남의 자료를 열어보는 경로를 없앱니다.
발행 먹등키	<code>external_content_id + external_version</code> 쌍이 Agent 발행물과의 연결점이자 먹등키입니다.
soft delete 통일	<code>deleted_at</code> 으로 통일했습니다(users-bookmarks-cards-reports-interests). 실수 삭제를 복구할 수 있고, 통계에서 과거를 잃지 않습니다.
형식은 값으로 전달	<code>reports.change_history_enabled</code> 는 그 본문이 어떤 형식인지 를 리포트마다 기록합니다. 계정 설정으로 화면을 그리면 사용자가 설정을 끄는 순간 과거 카드가 전부 깨지기 때문입니다.

공개 범위 결정 규칙

카드의 최초 공개 범위는 **세 단계로 판정**합니다. 사용자 설정을 그대로 따르지 않습니다.

순서	조건	결과	이유
1	아침 브리핑 유형	PRIVATE 강제	내 관심사를 그대로 드러내는 개인 브리핑입니다.
2	출처가 2건 미만	PRIVATE 강제	근거가 한 건뿐이면 남에게 보여줄 만큼 단단하지 않습니다.
3	그 외	사용자 기본값 (PUBLIC)	공개 피드가 채워지도록 V31에서 opt-out으로 전환했습니다.

근거 - `PublishProcessingService.initialVisibility().MIN_CITATIONS_FOR_PUBLIC=2`, `V17__user_settings.sql`, `V31__default_card_visibility_public.sql`. V31은 기본값과 기존 계정만 바꾸고 과거 카드는 소급 공개하지 않습니다.

3.2 agent-db 주요 테이블

영역	테이블
Job · 큐	agent_jobs, agent_job_attempts, event_inbox, event_outbox
발행	publish_snapshots, publish_attempts
사용자 원본	user_source_documents, user_source_document_versions, user_source_bindings
개인 위키	wiki_documents, wiki_document_versions, wiki_document_relations, wiki_chunks, wiki_embeddings, wiki_versions
관심사	user_interests, user_interest_profiles, interest_evidence, interest_taxonomy_*
전역 수집	global_sources, global_source_documents, global_collection_runs, global_trends
생성	generation_requests, generation_runs, generated_content_candidates, citations
변경점	change_history_runs, change_history_facts
브리핑	briefing_topic_selections, briefing_topic_snapshots
품질·안전	quality_evaluations, safety_evaluations
LLM 운영	model_configs, prompt_templates, usage_logs, provider_rate_limits, llm_batches
MCP·감사	api_keys, audit_logs

검색은 벡터만 쓰지 않습니다. `vector(1536)` 임베딩 유사도에 PostgreSQL `tsvector` 전문검색과 `pg_trgm` 트라이그램 인덱스를 함께 조합합니다. 고유명사처럼 임베딩이 약한 질의를 전문검색이 받아냅니다.

4. 주요 설계 결정과 근거

결정	왜 그렇게 했나	근거
브로커 없이 Outbox + Claim/Ack	단일 VM MVP에서 Kafka는 운영 비용이 이득을 넘습니다. DB 트랜잭션 안에서 Outbox를 쓰면 "저장은 됐는데 이벤트는 유실" 문제가 원천 차단됩니다.	GenerationScheduleOutboxWorker, generation_schedule_outbox
Pull(claim) 방향	Agent가 Service를 호출하지 않으면 Agent는 Service의 주소도 인증도 몰라도 됩니다. 경계가 단방향으로 단순해집니다.	PublishPollingWorker
FOR UPDATE SKIP LOCKED + lease	워커를 복제해도 중복 처리가 없습니다. replica를 1에서 10까지 올리면서 애플리케이션 코드는 0줄 수정했습니다. 같은 일을 나눠 하므로 LLM 비용도 늘지 않습니다.	docker-compose.yml:289
2 DB 물리 분리	팀 경계를 인프라로 강제합니다. 접속 정보를 주지 않는 방식이라 규칙이 문서가 아니라 구조입니다.	agent 컨테이너 env
한 이미지 다중 진입점	빌드 1회로 6개 역할을 배포합니다. 역할별로 이미지를 따로 만들면 버전이 어긋납니다.	compose command 오버라이드
max_connections =700	agent-db에 붙는 상주 프로세스가 28개이고 프로세스당 최대 16 커넥션이라 이론상 448이 필요합니다. 평시엔 훨씬 적지만 05시 아침 발화처럼 전 워커가 동시에 붙는 순간 이 정확히 상한을칩니다.	docker-compose.yml:135~143
zone="Asia/Seoul"	컨테이너 TZ와 무관하게 KST로 고정합니다. 과거 "UTC라 22시로 보정"한 커밋이 실제로는 밤 10시에 도는 사고가 있었습니다.	docker-compose.yml:88
진행률 바를 넣지 않음	몇 퍼센트인지 말할 근거가 없는데 숫자를 만들면 거짓말이 됩니다. 대신 완성물이 실제로 도착했을 때 슬롯을 닫습니다.	generation_pendings.card_public_id

5. 캐싱과 분산 상태 관리

Redis를 도입하지 않았습니다. 대신 **같은 목적을 DB와 무상태 설계로** 달성했습니다. 컴포넌트를 하나 덜 늘리는 쪽을 택한 결정입니다.

요구	우리 구현	근거
공유 캐시	<code>agent.global_source_documents</code> — URL의 SHA-256 앞 24자를 <code>url_key</code> (UNIQUE)로 쓰는 수집 캐시입니다. 한 번 읽은 본문을 워커 26개와 전체 사용자가 재사용 하므로 같은 기사에 외부 API를 두 번 호출하지 않습니다.	<code>0008_extract_global_source_cache.sql</code>
단기 캐시	외부 검색 응답에 90초 TTL 캐시, 정적 파생물에 프로세스 내 캐시를 씁니다.	<code>connectors/features/reddit.py</code>
중복 요청 방지	먹등기로 요청을 접고, <code>publish_snapshots</code> 에 결과를 저장해 재시도가 LLM을 다시 호출하지 않게 합니다.	<code>generation_pendings.idempotency_key</code>
세션 일관성	<code>SessionCreationPolicy.STATELESS</code> — 서버 세션을 만들지 않습니다. 동기화할 상태가 없으므로 인스턴스를 늘려도 세션 불일치가 발생하지 않습니다.	<code>SecurityConfig.java:61</code>
분산 락-조정	<code>FOR UPDATE SKIP LOCKED</code> + <code>lease</code> 로 워커 경쟁을 조정합니다. Redis 분산 락이 하는 일을 DB 트랜잭션으로 합니다.	<code>agent_jobs</code> , <code>publish_snapshots</code>
요청 제한	Nginx에서 IP별 <code>limit_req</code> — <code>/oauth</code> 5r/s, <code>/mcp</code> 10r/s, 초과 시 429.	<code>nginx.conf:14~15</code>

응답 캐시는 넣지 않았고, 그 이유를 측정했습니다.

Spring `@Cacheable` 0건, Nginx `proxy_cache_path` 미설정 — 응답 캐시 계층은 없습니다. 2026-08-12 부하 테스트에서 읽기 경로의 병목은 4 vCPU CPU 포화였고, 캐시는 방향이 맞습니다. 다만 **히트율이 나올 트래픽 규모가 아니고**(실 사용자-공개 카드가 각각 두 자리), 같은 측정에서 **트랜잭션이 커넥션을 쥔 채 CPU 작업을 하고 있는 것**이 먼저 나왔습니다.

캐시가 성립하는 경로는 확인해 뒀습니다 — 게스트 공개 피드는 개인화를 적용하지 않아

(`FeedService.rankForViewer()`가 뷰어 ID 없으면 최신순 그대로 반환) 모든 방문자에게 동일한 응답입니다. 넣는다면 ① 게스트 공개 피드에 짧은 TTL(회전 슬롯 300초보다 짧게) ② 준정적 데이터 ③ 사용자별 캐시 순서입니다. **"안했다"가 아니라 "순서를 정했다"가 정확합니다.**

6. 로컬 환경과 배포 환경의 차이

	로컬	배포(운영 실측 2026-08-13)
이미지	소스 직접 실행	GHCR :latest pull
Nginx · HTTPS	없음 (직접 포트 접근)	있음 — 유일한 공개 경계 / Let's Encrypt
Agent 연동	mock (인프로세스 큐)	http — 실제 Agent 경로
생성 스케줄러	false	true — 03 → 05 → 07시
Agent 워커	없음	26개 (report 10 · briefing 8 · wiki 6 · url 1 · global 1)

로컬 기본값이 전부 mock입니다. 그래서 로컬에서 잘 돈다 ≠ 배포에서 실제 Agent가 돈다고, 이 차이가 과거 장애의 반복 원인이었습니다. 기능 검증은 배포 환경 기준으로 합니다.

7. 알려진 위험과 남은 과제

항목	상태	영향과 조치
개발용 호스트 포트 2개	미제거	agent-db (5432)와 agent-api (8000)가 호스트로 열려 있습니다. Compose 주식에 "운영 전 제거"가 적혀 있으나 아직 남았습니다. Agent 내부 API는 Bearer 토큰이 걸려 있어 무인 증 노출은 문서 경로뿐이지만, 내부 전용 원칙과 어긋나므로 달아야 합니다.
1 DB 시절 잔재	미사용 확정	service-db에 agent.generation_logs · prompt_templates · retrieval_logs 3개가 남아 있습니다. Java 코드 참조 0건 으로 확인했습니다. 제출 후 정리 대상입니다.
미사용 환경변수	정리 필요	Compose의 GDELT_API_KEY 는 코드에서 읽지 않습니다(GDELT는 키가 필요 없는 API).
응답 캐시	계획 수립	위 5장의 순서대로, 트래픽이 히트율을 만들 시점에 게스트 피드부터 도입합니다.

부록. 이번 검증에서 바로잡은 값

이전 설계 자료와 코드를 대조해 아래 값을 수정했습니다. 왼쪽이 기존 기재값입니다.

항목	기존 기재	실제 (2026-08-13 main)
agent-db 커넥션 상한	200	700 (요구 448)
커넥션 계산 근거	8프로세스 × 16 = 128	28프로세스 × 16 = 448
리포트 워커	×3	×10
브리핑 워커	다이어그램·표에 누락	×8 (워커 5종 · 총 26개)
Compose 서비스 정의	14종	15종 (상시 기동 34개는 일치)
생성 트리거 시각	07:00 cron	05:00 생성 / 07:00 공개
Flyway 마이그레이션	28개	30개 (최신 V31)
agent 마이그레이션	28개	32개 (최신 0032)
agent-db 테이블	64개	65개 (관리 테이블 포함)
카드 기본 공개범위	기본 PRIVATE	기본 PUBLIC — 단 아침 브리핑·출처 2건 미만은 PRIVATE 강제
외부 API	4개	6개 (GDELT·NewsAPI 누락되어 있었음)
Service 워커	2개 (Outbox·Publish)	4개 (+ 스크랩 워커 · Job 상태)
클리퍼 버전	v0.4.2	v0.4.1
캐싱	"캐시 계층 없음"	응답 캐시는 없으나 수집 공유 캐시·단기 캐시는 있음
피드 후보 풀 개선	"60→20으로 줄여 처리량 +62%"	코드는 여전히 60 — 미반영. 측정에서 확인했으나 main에 들어오지 않았습니다.

마지막 두 줄이 이번 검증의 핵심입니다. **운영이 정상이라는 게 설정과 문서가 맞다는 뜻은 아닙니다.** 손으로 고친 값이 코드로 돌아오지 않으면 다음 배포까지만 사는 설정이고, 측정으로 얻은 개선도 반영되지 않으면 문서에만 존재하는 성과입니다. 같은 종류의 어긋남을 막기 위해 워커 수와 커넥션 상한은 **CI 단정문으로 고정했습니다**(bambi-build/.github/workflows/ci.yml).

